UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT

Data Structure and Algorithm

Laboratory Activity No. 14

Tree Structure Analysis

Submitted by:
Aquino, Jester J.

Instructor:
Engr. Maria Rizette H. Sayo

November 9, 2025

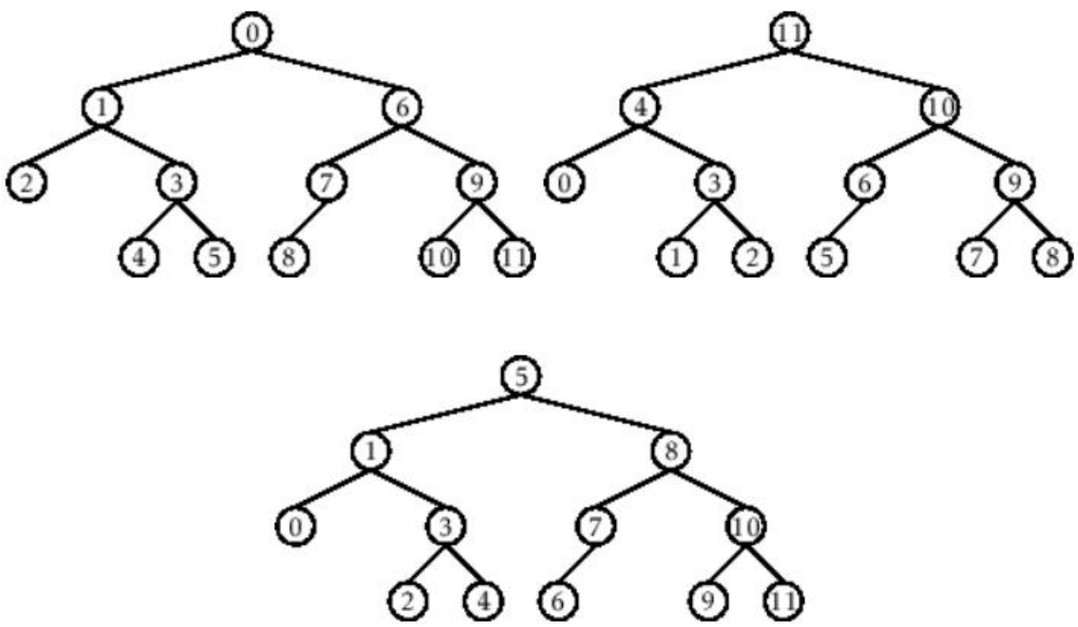
I. Objectives

Introduction

An abstract non-linear data type with a hierarchy-based structure is a tree. It is made up of links connecting nodes (where the data is kept). The root node of a tree data structure is where all other nodes and subtrees are connected to the root.

This laboratory activity aims to implement the principles and techniques in:

- To introduce Tree as Non-linear data structure
- To implement pre-order, in-order, and post-order of a binary tree



- Figure 1. Pre-order, In-order, and Post-order numberings of a binary tree

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Tree Implementation

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []

    def add_child(self, child_node):
        self.children.append(child_node)

    def remove_child(self, child_node):
        self.children = [child for child in self.children if child != child_node]
```

```
    def traverse(self):
        nodes = [self]
        while nodes:
            current_node = nodes.pop()
            print(current_node.value)
            nodes.extend(current_node.children)

    def __str__(self, level=0):
        ret = "  " * level + str(self.value) + "\n"
        for child in self.children:
            ret += child.__str__(level + 1)
        return ret

# Create a tree
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

print("Tree structure:")
print(root)

print("\nTraversal:")
root.traverse()
```

Questions:

1. What is the main difference between a binary tree and a general tree?
2. In a Binary Search Tree, where would you find the minimum value? Where would you find the maximum value?
3. How does a complete binary tree differ from a full binary tree?
4. What tree traversal method would you use to delete a tree properly? Modify the source codes.

III. Results

1. What is the main difference between a binary tree and a general tree?

Answer: Binary Tree has each node has at most two children, Commonly used in algorithm like Binary Search Trees and also has specific traversal orders while General Tree each node can have any number of children, Used for hierarchical data like file system, organization charts, Traversal is usually depth-first or depth-first.

2. In a Binary Search Tree, where would you find the minimum value? Where would you find the maximum value?

Answer: In Binary Search Tree you can find the minimum value by following the leftmost child from the root while the maximum value found by following the rightmost child from the root.

3. How does a complete binary tree differ from a full binary tree?

Answer: Full binary tree every node has 0 or 2 children no node has only one child while the Complete binarry tree all levels are completely filled except possibly the last, which is filled from left to right.

4. What tree traversal method would you use to delete a tree properly? Modify the source codes.

Answer: For me I would use postorder traversal because you must delete children first before the parent node, This prevent losing references to children before they are deleted

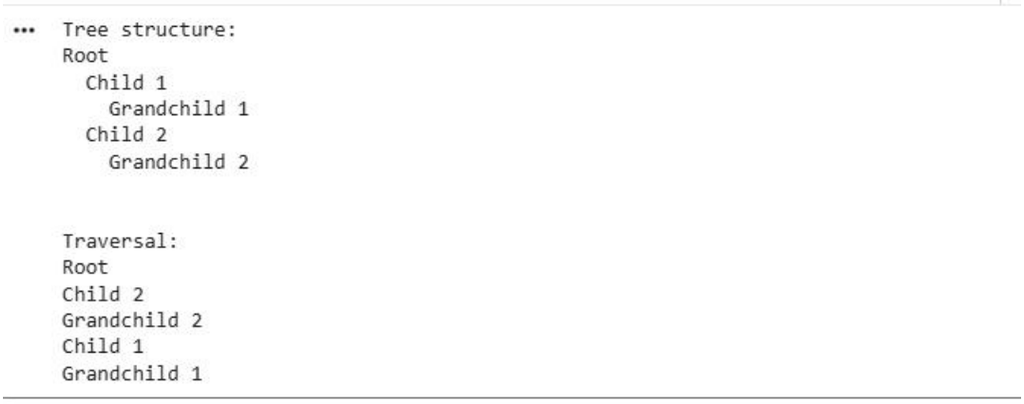


Figure 1 Screenshot of program

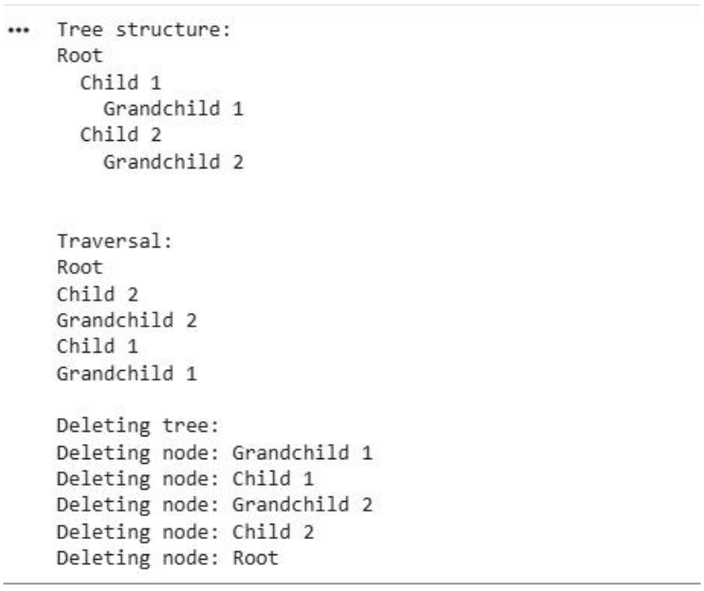


Figure 2 Screenshot of program

IV. Conclusion

In this activity, we explored the structure and behavior of trees in Python, focusing on how to create, traverse, and delete nodes within a general tree. We learned that a binary tree restricts each node to at most two children, while a general tree allows multiple children per node. We also understood that in a Binary Search Tree (BST), the smallest value is located at the leftmost node, and the largest value at the rightmost node. Moreover, the distinction between a complete and a full binary tree was highlighted, showing how they differ in structure and node arrangement.

By implementing a postorder traversal for deleting a tree, we learned the importance of removing child nodes before their parent nodes to ensure proper memory cleanup and prevent data loss. Overall, this activity deepened our understanding of tree data structures and their essential operations in programming.

References

[1] GeeksforGeeks. (2024, March 15). Tree data structure. GeeksforGeeks. https://www.geeksforgeeks.org/tree-data-structure/